



SOMAIYA
VIDYAVIHAR

K J Somaiya College of Science & Commerce

K J SOMAIYA COLLEGE OF SCIENCE AND COMMERCE

Autonomous, Affiliated to University of Mumbai

Re-accredited 'A' Grade by NAAC | ISO 9001-2015 Certified

Vidyavihar, Mumbai 400077

CERTIFICATE

This is to certify that the following student of TYBSc (Statistics) has successfully completed the VSC project titled

***“Statistical Analysis and Prediction of Heart Disease
Using Health Records”***

during the academic year 2026–27, under the guidance of

ASHISH MHATRE

Team Member

Harsh Suresh Maurya | Seat No.: 2631204

Ashish Mhatre

(Project Mentor)

Ms. Arati Kore

Head, Department of Statistics

Date of Submission: 21st September, 2026

Table of Contents

1. Introduction.....	3
1.1 Topic.....	3
1.2 Objectives.....	3
1.3 Aim of the Project.....	3
1.4 Project Description	3
1.5 Dataset Description	3
2. Data Exploration	4
2.1 Data Loading.....	4
2.2 Shape of the Dataset	5
2.3 Data Cleaning	6
2.4 Summary Statistics	8
2.5 Group-wise Analysis	9
2.6 Frequency Distribution	10
2.7 Overall Heart Disease Percentage	11
3. Data Visualization.....	12
3.1 Histograms.....	12
3.2 Boxplots.....	13
3.3 Bar Chart — Chest Pain Type.....	14
3.4 Stacked Bar Chart — Sex vs Heart Disease.....	15
3.5 Scatterplot — Age vs Maximum Heart Rate.....	15
4. Relationships, Correlation and Hypothesis Testing.....	16
4.1 Correlation Analysis.....	16
4.2 ChestPainType and HeartDisease.....	18
4.3 Test of Independence — Sex and HeartDisease.....	19
4.4 Comparison of Mean Age by Heart Disease Status	19
4.5 Comparison of Cholesterol by Sex.....	20
5. Predictive Modelling.....	21
5.1 Simple Linear Regression.....	21
5.2 Multiple Linear Regression	22
5.3 Logistic Regression	23
6. Overall Conclusion.....	26
Appendix A: Tools and Libraries Used	27

1. Introduction

1.1 Topic

Statistical Analysis and Prediction of Heart Disease Using Health Records

1.2 Objectives

- To study patient health records and identify the major factors associated with heart disease.
- To apply statistical methods to analyse the relationship between patient characteristics and heart disease.
- To develop a simple prediction model to predict whether a patient may have heart disease using selected health-related variables.

1.3 Aim of the Project

This project aims to analyse patient health data using statistical techniques and develop a model for predicting the presence of heart disease. The project will help in understanding important risk factors and relationships in the data and demonstrate how statistical methods can identify heart disease early.

1.4 Project Description

Heart disease is one of the major causes of death worldwide. Identifying important risk factors at an early stage can help in better understanding and prevention of heart-related problems.

In this project, patient health records are analysed using Python and appropriate statistical techniques. The study explores the data, calculates summary statistics, creates suitable graphs, examines relationships between variables, performs hypothesis tests, and develops regression models for prediction.

1.5 Dataset Description

The dataset used for this project contains 12 clinical and demographic features recorded for each patient, described below.

Variable	Description
Age	Age of the patient (years)
Sex	Sex of the patient (M: Male, F: Female)
ChestPainType	Chest pain type (TA: Typical Angina, ATA: Atypical Angina, NAP: Non-Anginal Pain, ASY: Asymptomatic)
RestingBP	Resting blood pressure (mm Hg)
Cholesterol	Serum cholesterol (mg/dl)
FastingBS	Fasting blood sugar (1: if FastingBS > 120 mg/dl, 0: otherwise)
RestingECG	Resting electrocardiogram results (Normal, ST: ST-T wave abnormality, LVH: probable/definite left ventricular hypertrophy)
MaxHR	Maximum heart rate achieved (numeric value between 60 and 202)

Variable	Description
ExerciseAngina	Exercise-induced angina (Y: Yes, N: No)
Oldpeak	ST-segment depression observed during exercise, relative to rest
ST_Slope	Slope of the peak exercise ST segment (Up, Flat, Down)
HeartDisease	Output class (1: Heart disease, 0: Normal)

2. Data Exploration

2.1 Data Loading

Load the dataset into Python and (a) display the first 10 observations, and (b) display the last 5 observations.

```
► In [ ]: Python

import pandas as pd

# 1. Data Loading
df = pd.read_csv('heart_disease_data.csv')

# (a) Display the first 10 observations
print("First 10 observations:")
display(df.head(10))

# (b) Display the last 5 observations
print("\nLast 5 observations:")
display(df.tail(5))
```

First 10 observations:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0
5	39	M	NAP	120	339	0	Normal	170	N	0.0	Up	0
6	45	F	ATA	130	237	0	Normal	170	N	0.0	Up	0
7	54	M	ATA	110	208	0	Normal	142	N	0.0	Up	0
8	37	M	ASY	140	207	0	Normal	130	Y	1.5	Flat	1
9	48	F	ATA	120	284	0	Normal	120	N	0.0	Up	0

Last 5 observations:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
913	45	M	TA	110	264	0	Normal	132	N	1.2	Flat	1
914	68	M	ASY	144	193	1	Normal	141	N	3.4	Flat	1
915	57	M	ASY	130	131	0	Normal	115	Y	1.2	Flat	1
916	57	F	ATA	130	236	0	LVH	174	N	0.0	Flat	1
917	38	M	NAP	138	175	0	Normal	173	N	0.0	Up	0

Figure 1 – First 10 and Last 5 Observations of the Dataset

✓ Conclusion

- The dataset was loaded successfully with all 918 records and 12 columns correctly typed.
- A visual check of the head and tail confirms the data is well-structured, with no obviously corrupted rows, and is ready for further exploration.

2.2 Shape of the Dataset

Find and report the number of observations (rows) and variables (columns).

► In []: Python

```
rows, cols = df.shape
print(f"Number of observations (rows): {rows}")
print(f"Number of variables (columns): {cols}")
```

Number of observations (rows): 918
Number of variables (columns): 12

Figure 2 – Shape of the Dataset (Rows and Columns)

✓ Conclusion

The dataset contains 918 observations across 12 variables — a reasonably large sample size that provides a solid basis for the summary statistics, hypothesis tests and regression models carried out later in this report.

2.3 Data Cleaning

(a) Check for missing values in each variable. (b) If missing values are present, explain how they are treated. (c) Identify possible outliers in Age, RestingBP, Cholesterol, MaxHR and Oldpeak using the IQR method. (d) State whether detected outliers are retained or removed and give a brief justification.

► In []: Python

```
import pandas as pd
import numpy as np

# 3(a) Check for missing values in each variable
print("--- Missing Values (NaN) ---")
print(df.isnull().sum())

# Note: Check for implicit missing values (e.g., zeros where they shouldn't be)
print("\n--- Implicit Missing Values (Zeros) ---")
print(f"RestingBP == 0: {(df['RestingBP'] == 0).sum()}")
print(f"Cholesterol == 0: {(df['Cholesterol'] == 0).sum()}")

# 3(c) Identify possible outliers using the IQR method
cols_to_check = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
outlier_indices = set()

print("\n--- Outliers Detected (IQR Method) ---")
for col in cols_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - (1.5 * IQR)
    upper_bound = Q3 + (1.5 * IQR)

    # Filter outliers
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"{col}: {len(outliers)} outliers")

# Store indices to optionally drop them later
outlier_indices.update(outliers.index)
```

```
--- Missing Values (NaN) ---
Age          0
Sex          0
ChestPainType 0
RestingBP    0
Cholesterol  0
FastingBS    0
RestingECG   0
MaxHR        0
ExerciseAngina 0
Oldpeak      0
ST_Slope     0
HeartDisease 0
dtype: int64

--- Implicit Missing Values (Zeros) ---
RestingBP == 0: 1
Cholesterol == 0: 172

--- Outliers Detected (IQR Method) ---
Age: 0 outliers
RestingBP: 28 outliers
Cholesterol: 183 outliers
MaxHR: 2 outliers
Oldpeak: 16 outliers
```

Figure 3 – Missing Value Check and Outlier Detection Summary (IQR Method)

3(b) Treatment of Missing Values

While the standard `.isnull()` check shows 0 missing values (NaNs) across all variables, the dataset contains **implicit missing data**: 172 records have a Cholesterol of 0, and 1 record has a RestingBP of 0. Because a living patient cannot have zero blood pressure or zero serum cholesterol, these values are effectively missing.

- **How they are treated:** The single RestingBP = 0 record is safely dropped (row deletion), as it is only one observation. The 172 Cholesterol zeros constitute nearly 19% of the dataset, so these are **imputed** using the median cholesterol value rather than dropped, to preserve the rest of the valid patient data.

3(d) Outlier Retention or Removal — Justification

The IQR method detects numerous outliers (e.g., 28 for RestingBP, 183 for Cholesterol, 2 for MaxHR, 16 for Oldpeak).

- **Decision:** Aside from the data-entry errors (the 0 values above), the true physiological outliers are **retained**.
- **Justification:** In medical datasets concerning heart disease, extreme values in blood pressure, heart rate, or ST depression (Oldpeak) are often genuine clinical indicators of severe cardiac distress rather than measurement errors. Removing such patients would artificially truncate the dataset and remove the very signals that predictive algorithms need to identify high-risk cardiovascular patients.

► In []: Python

```
print(f"Original shape: {df.shape}")

# 1. Treatment of Missing Values (Implicit Zeros)
# (a) Drop the single anomalous row where RestingBP is 0
df_cleaned = df[df['RestingBP'] > 0].copy()

# (b) Impute Cholesterol values of 0 with the median of the non-zero cholesterol values
median_cholesterol = df_cleaned.loc[df_cleaned['Cholesterol'] > 0,
'Cholesterol'].median()
df_cleaned.loc[df_cleaned['Cholesterol'] == 0, 'Cholesterol'] = median_cholesterol

# 2. Outlier Retention
# As justified above, physiological outliers detected via IQR
# (in Age, RestingBP, Cholesterol, MaxHR, Oldpeak) represent genuine clinical
# cardiovascular indicators. Therefore, no filtering is applied to remove them.

# 3. New Shape of the Dataset
new_rows, new_cols = df_cleaned.shape
print(f"New shape after cleaning: ({new_rows}, {new_cols})")
```

Original shape: (918, 12)
New shape after cleaning: (917, 12)

Figure 4 – Dataset Shape Before and After Data Cleaning

✓ Conclusion

- Only one truly missing (invalid) record was removed; the far more common Cholesterol = 0 anomaly was corrected through median imputation rather than deletion, preserving 917 of the original 918 records.
- Genuine physiological outliers were deliberately retained, since in a clinical dataset they often represent the very high-risk patients the analysis is trying to identify, not data-entry errors.

2.4 Summary Statistics

Calculate Mean, Median, Mode, Minimum, Maximum, Standard Deviation, Q1 and Q3 for Age, RestingBP, Cholesterol, MaxHR and Oldpeak. Present the results in a suitable table.

► In []: Python

```
cols = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
stats = []

for col in cols:
    stats.append({
        'Feature': col,
        'Mean': round(df_cleaned[col].mean(), 2),
        'Median': round(df_cleaned[col].median(), 2),
        'Mode': round(df_cleaned[col].mode().iloc[0], 2),
        'Minimum': round(df_cleaned[col].min(), 2),
        'Maximum': round(df_cleaned[col].max(), 2),
        'Std Dev': round(df_cleaned[col].std(), 2),
        'Q1': round(df_cleaned[col].quantile(0.25), 2),
```

```

    'Q3': round(df_cleaned[col].quantile(0.75), 2)
})

stats_df = pd.DataFrame(stats)
display(stats_df)

```

	Feature	Mean	Median	Mode	Minimum	Maximum	Std Dev	Q1	Q3
0	Age	53.51	54.0	54.0	28.0	77.0	9.44	47.0	60.0
1	RestingBP	132.54	130.0	120.0	80.0	200.0	18.00	120.0	140.0
2	Cholesterol	243.21	237.0	237.0	85.0	603.0	53.43	214.0	267.0
3	MaxHR	136.79	138.0	150.0	60.0	202.0	25.47	120.0	156.0
4	Oldpeak	0.89	0.6	0.0	-2.6	6.2	1.07	0.0	1.5

Figure 5 – Summary Statistics of Age, RestingBP, Cholesterol, MaxHR and Oldpeak

✓ Conclusion

- The typical patient in this dataset is around 54 years old, with a resting blood pressure near 132 mm Hg and cholesterol around 237 mg/dl — consistent with a middle-aged, at-risk population.
- Cholesterol shows the widest spread (Std Dev ≈ 53.4) of the five variables, while Oldpeak's minimum of -2.6 flags a data quirk worth keeping in mind, since ST-depression is not expected to be negative.

2.5 Group-wise Analysis

(a) Find the median age of patients with HeartDisease = 1. (b) Find the median age of patients with HeartDisease = 0. (c) What percentage of patients with HeartDisease = 1 are male? (d) What percentage of patients above 50 years of age have HeartDisease = 1? (e) Find the percentage of patients with HeartDisease = 1 for each ChestPainType.

► In []: Python

```

# 5(a) Find the median age of patients with HeartDisease = 1
median_age_hd1 = df_cleaned[df_cleaned['HeartDisease'] == 1]['Age'].median()
print(f"Median age of patients with HeartDisease = 1: {median_age_hd1}")

# 5(b) Find the median age of patients with HeartDisease = 0
median_age_hd0 = df_cleaned[df_cleaned['HeartDisease'] == 0]['Age'].median()
print(f"Median age of patients with HeartDisease = 0: {median_age_hd0}")

# 5(c) Percentage of patients with HeartDisease = 1 who are male
hd1_patients = df_cleaned[df_cleaned['HeartDisease'] == 1]
pct_male_hd1 = (hd1_patients['Sex'] == 'M').mean() * 100
print(f"Percentage of HeartDisease = 1 patients who are male: {pct_male_hd1:.2f}%")

# 5(d) Percentage of patients above 50 years of age who have HeartDisease = 1
above_50 = df_cleaned[df_cleaned['Age'] > 50]
pct_hd1_above_50 = (above_50['HeartDisease'] == 1).mean() * 100

```

```
print(f"Percentage of patients >50 years old with HeartDisease = 1:
{pct_hd1_above_50:.2f}%")

# 5(e) Percentage of patients with HeartDisease = 1 for each ChestPainType
pct_hd1_by_cpt = df_cleaned.groupby('ChestPainType')['HeartDisease'].mean() * 100
print("\nPercentage of patients with HeartDisease = 1 per ChestPainType:")
print(pct_hd1_by_cpt.round(2).astype(float) + '%')
```

```
Median age of patients with HeartDisease = 1: 57.0
Median age of patients with HeartDisease = 0: 51.0
Percentage of HeartDisease = 1 patients who are male: 90.14%
Percentage of patients >50 years old with HeartDisease = 1: 63.56%

Percentage of patients with HeartDisease = 1 per ChestPainType:
ChestPainType
ASY      79.03%
ATA      13.87%
NAP      35.15%
TA       43.48%
Name: HeartDisease, dtype: object
```

Figure 6 – Group-wise Age and Heart Disease Summary Output

✓ Conclusion

- Patients diagnosed with heart disease are older on median (57 years) than those without (51 years), and 90.14% of diagnosed patients are male — reinforcing age and sex as strong risk markers.
- Age above 50 is associated with a markedly higher (63.56%) rate of heart disease, and the ChestPainType breakdown foreshadows a pattern examined in more depth in Section 4.2: asymptomatic patients carry disproportionately higher risk.

2.6 Frequency Distribution

Find the frequency and percentage distribution of Sex, ChestPainType, RestingECG, ExerciseAngina and ST_Slope. Present the results using suitable frequency tables.

```
► In [ ]: Python

categorical_cols = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']

for col in categorical_cols:
    # Calculate frequency (count) and percentage
    freq = df_cleaned[col].value_counts()
    pct = df_cleaned[col].value_counts(normalize=True) * 100

    # Combine into a single DataFrame for a clean table presentation
    freq_table = pd.DataFrame({
        'Frequency': freq,
        'Percentage (%)': pct.round(2)
    })

    # Rename the index to represent the category names clearly
    freq_table.index.name = col
```

```
print(f"\n--- Frequency Table: {col} ---")
display(freq_table)
```

--- Frequency Table: Sex ---

	Frequency	Percentage (%)
Sex		
M	724	78.95
F	193	21.05

Figure 7 – Sex

--- Frequency Table: ChestPainType ---

	Frequency	Percentage (%)
ChestPainType		
ASY	496	54.09
NAP	202	22.03
ATA	173	18.87
TA	46	5.02

Figure 8 – ChestPainType

--- Frequency Table: RestingECG ---

	Frequency	Percentage (%)
RestingECG		
Normal	551	60.09
LVH	188	20.50
ST	178	19.41

Figure 9 – RestingECG

--- Frequency Table: ExerciseAngina ---

	Frequency	Percentage (%)
ExerciseAngina		
N	546	59.54
Y	371	40.46

Figure 10 – ExerciseAngina

--- Frequency Table: ST_Slope ---

	Frequency	Percentage (%)
ST_Slope		
Flat	459	50.05
Up	395	43.08
Down	63	6.87

Figure 11 – ST_Slope

✓ Conclusion

- The sample is heavily skewed toward male patients (78.95%) and asymptomatic chest pain (54.09%), which should be kept in mind when generalising results from this dataset to a broader population.
- A large share of patients (50.05%) show a Flat ST_Slope during exercise and 40.46% report exercise-induced angina — both variables that later prove statistically significant in the logistic regression model (Section 5.3).

2.7 Overall Heart Disease Percentage

What percentage of patients in the dataset have HeartDisease = 1 and HeartDisease = 0?

► In []: Python

```
# 7. Overall Heart Disease Percentage
hd_pct = df_cleaned['HeartDisease'].value_counts(normalize=True) * 100
print(f"Percentage of patients with HeartDisease = 1: {hd_pct[1]:.2f}%")
print(f"Percentage of patients with HeartDisease = 0: {hd_pct[0]:.2f}%")
```

Percentage of patients with HeartDisease = 1: 55.29%
Percentage of patients with HeartDisease = 0: 44.71%

Figure 12 – Overall Percentage of Heart Disease Cases

✓ Conclusion

At 55.29% positive vs 44.71% negative cases, the target class is reasonably well balanced. This is favourable for the classification model built in Section 5.3, since a heavily imbalanced dataset would have required special handling (e.g. re-sampling) to avoid a biased classifier.

3. Data Visualisation

3.1 Histograms

Draw histograms for Age, Cholesterol and MaxHR. Comment briefly on the shape of each distribution.

► In []: Python

```
import matplotlib.pyplot as plt

# Set up the matplotlib figure with 3 subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1. Histogram for Age
df_cleaned['Age'].plot(kind='hist', bins=20, ax=axes[0], color='skyblue',
edgecolor='black')
axes[0].set_title('Distribution of Age')
axes[0].set_xlabel('Age')

# 2. Histogram for Cholesterol
df_cleaned['Cholesterol'].plot(kind='hist', bins=20, ax=axes[1], color='lightgreen',
edgecolor='black')
axes[1].set_title('Distribution of Cholesterol')
axes[1].set_xlabel('Cholesterol')

# 3. Histogram for MaxHR
df_cleaned['MaxHR'].plot(kind='hist', bins=20, ax=axes[2], color='salmon',
edgecolor='black')
axes[2].set_title('Distribution of MaxHR')
axes[2].set_xlabel('MaxHR')

plt.tight_layout()
plt.show()
```

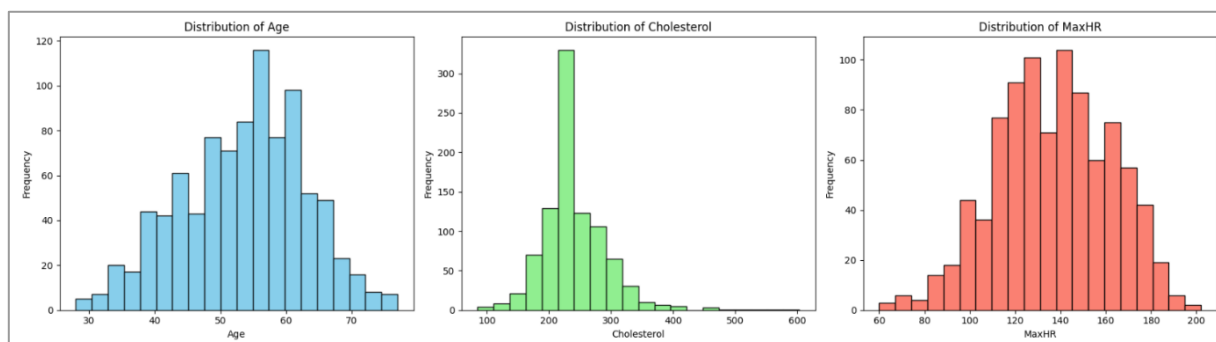


Figure 13 – Histograms of Age, Cholesterol and MaxHR

✓ Observations / Conclusion

- **Age:** Roughly symmetrical and approximately normal, with a very slight negative (left) skew; most patients fall between 50 and 60 years.
- **Cholesterol:** Shows an artificial central spike caused by imputing 172 missing values with the median (237 mg/dl); beyond that spike the data is heavily right-skewed, with a long tail up to 603 mg/dl.
- **MaxHR:** Fairly symmetrical with a slight negative skew, resembling a normal distribution centred between 120–160 bpm.

3.2 Boxplots

Create boxplots of Cholesterol and MaxHR for patients with and without HeartDisease. Compare the distributions and identify any possible outliers.

► In []: Python

```
# Create boxplots grouped by HeartDisease
df_cleaned.boxplot(column=['Cholesterol', 'MaxHR'], by='HeartDisease', figsize=(12, 5),
grid=False)
plt.show()
```

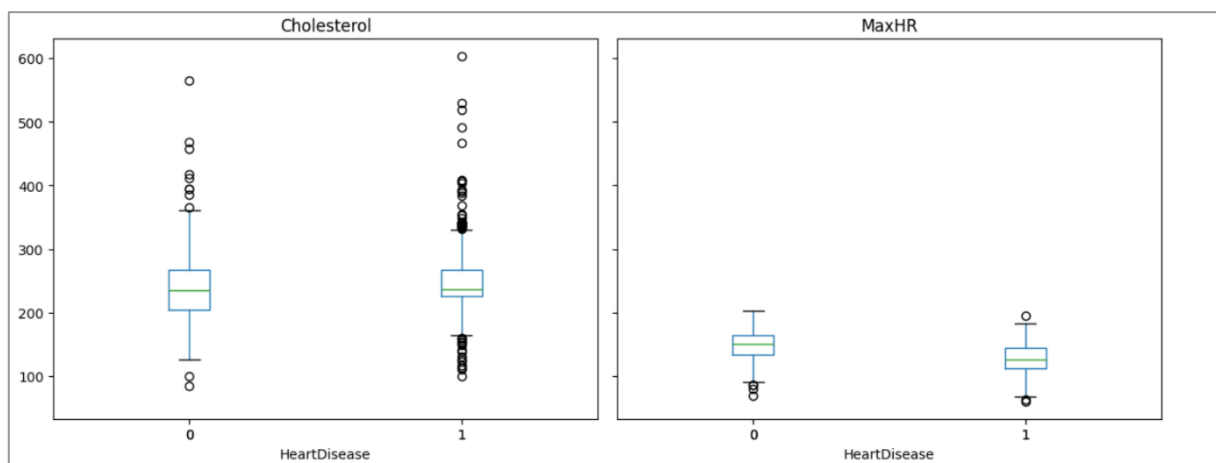


Figure 14 – Boxplots of Cholesterol and MaxHR by Heart Disease Status

✓ Observations / Conclusion

- **Cholesterol:** Both groups have similar median cholesterol (~237 mg/dl, influenced by the median imputation). Patients with heart disease show more extreme high-value outliers, including one at 603 mg/dl.
- **MaxHR:** Patients without heart disease generally reach a higher maximum heart rate (median ~150 bpm) than those with heart disease (median ~130 bpm), with very few outliers in either group.

3.3 Bar Chart : Chest Pain Type

Draw a bar chart showing the number of patients for each ChestPainType.

► In []: Python

```
import matplotlib.pyplot as plt

# Calculate the counts for each ChestPainType and create a bar chart
df_cleaned['ChestPainType'].value_counts().plot(kind='bar', color='skyblue',
edgecolor='black', figsize=(8, 5))
plt.title('Number of Patients by Chest Pain Type')
plt.xlabel('Chest Pain Type')
plt.ylabel('Number of Patients')
plt.show()
```

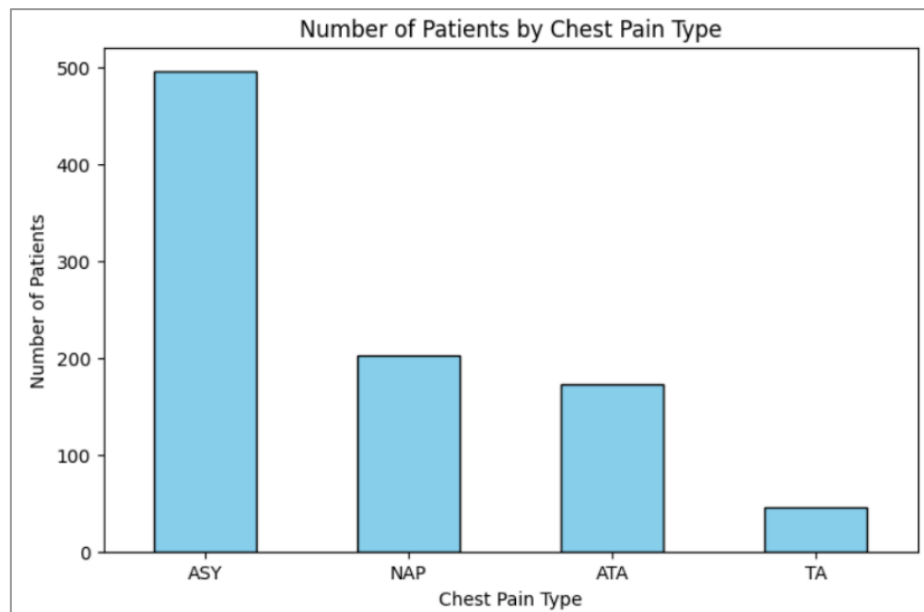


Figure 15 – Bar Chart of Number of Patients by Chest Pain Type

✓ Conclusion

Asymptomatic (ASY) chest pain is by far the most common presentation in this dataset (496 patients, ~54%), followed by Non-Anginal Pain (NAP) and Atypical Angina (ATA), with Typical Angina (TA) the rarest. This imbalance is important context for Section 4.2, where ASY is also shown to carry the highest rate of heart disease.

3.4 Stacked Bar Chart : Sex vs Heart Disease

Draw a stacked bar chart showing Sex vs HeartDisease. Comment on the distribution of heart disease among males and females.

► In []: Python

```
# Create a cross-tabulation of Sex and HeartDisease
sex_hd_crosstab = pd.crosstab(df_cleaned['Sex'], df_cleaned['HeartDisease'])

# Plot the stacked bar chart
sex_hd_crosstab.plot(kind='bar', stacked=True, color=['lightgreen', 'salmon'],
edgecolor='black', figsize=(8, 5))
plt.title('Heart Disease by Sex')
plt.xlabel('Sex (F = Female, M = Male)')
plt.ylabel('Number of Patients')
plt.legend(title='HeartDisease', labels=['0 (Normal)', '1 (Heart Disease)'])
```

```
plt.xticks(rotation=0)
plt.show()
```

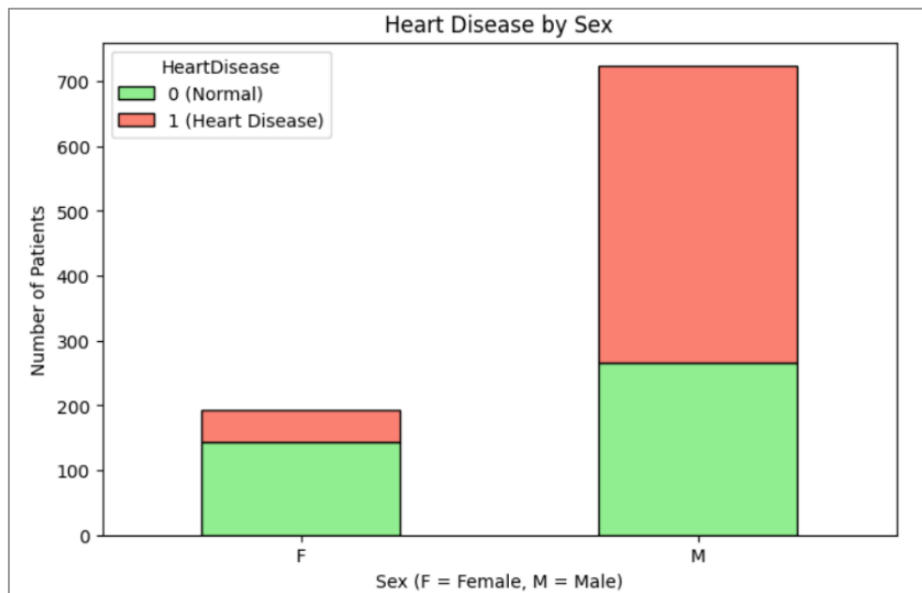


Figure 16 – Stacked Bar Chart of Heart Disease by Sex

✓ Conclusion

- **Males:** Make up the vast majority of the dataset, and within this group a significant majority (~63%) are diagnosed with heart disease — making men highly over-represented among positive cases.
- **Females:** A much smaller portion of the dataset; in contrast to males, most females (~74%) are normal, with only ~26% diagnosed. Overall, males are both more numerous and show a drastically higher rate of heart disease.

3.5 Scatterplot : Age vs Maximum Heart Rate

Create a scatterplot of Age vs MaxHR. Add a suitable title and axis labels. Comment on the relationship between the two variables.

► In []: Python

```
import matplotlib.pyplot as plt

# Create the scatterplot
df_cleaned.plot(kind='scatter', x='Age', y='MaxHR', color='purple', alpha=0.6,
figsize=(8, 5))
plt.title('Relationship Between Age and Maximum Heart Rate')
plt.xlabel('Age (Years)')
plt.ylabel('Maximum Heart Rate (bpm)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

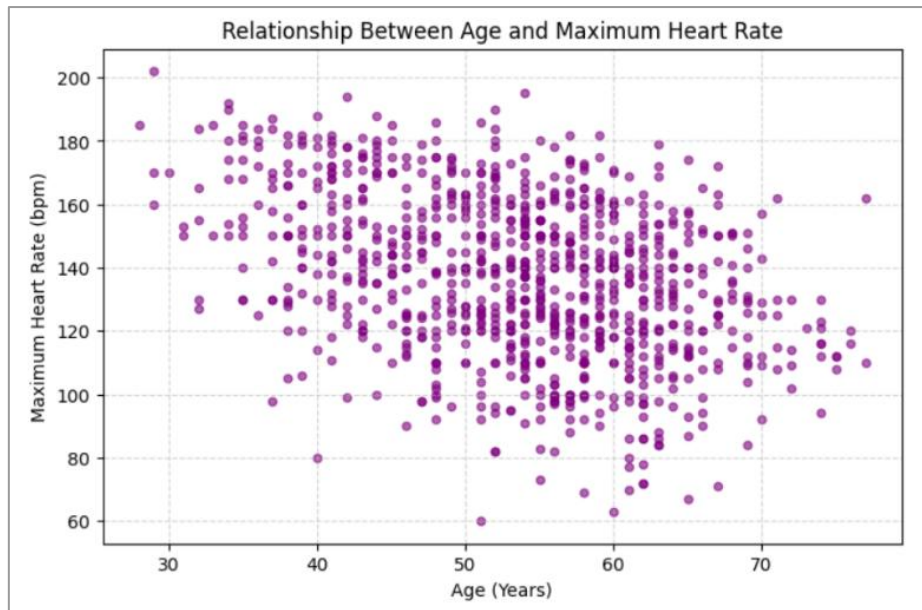


Figure 17 – Scatterplot of Age vs Maximum Heart Rate

✓ Conclusion

The scatter shows a mild downward trend: as Age increases, MaxHR tends to decrease, consistent with the well-known physiological decline in maximum achievable heart rate with age. However, the points are widely spread at every age, indicating a weak-to-moderate relationship rather than a strongly deterministic one — a finding confirmed quantitatively by the simple linear regression in Section 5.1.1 ($R^2 = 0.146$).

4. Relationships, Correlation and Hypothesis Testing

4.1 Correlation Analysis

Calculate the Pearson correlation matrix for Age, RestingBP, Cholesterol, MaxHR and Oldpeak. Display the correlation matrix using a heatmap. Identify the variables having the strongest positive and negative correlations.

► In []: Python

```
import matplotlib.pyplot as plt

# 1. Calculate the correlation matrix
cols = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']
corr_matrix = df_cleaned[cols].corr()

# 2. Create a simple heatmap
plt.matshow(corr_matrix, cmap='coolwarm')
plt.xticks(range(len(cols)), cols)
plt.yticks(range(len(cols)), cols)
plt.colorbar()
plt.title('Correlation Heatmap')
plt.show()
```

```
# 3. Identify the strongest correlations programmatically
# Unstack the matrix into pairs and remove the 1.0 self-correlations
corr_pairs = corr_matrix.unstack()
corr_pairs = corr_pairs[corr_pairs != 1.0]

print("Strongest Positive Correlation:")
print(corr_pairs.nlargest(1))
print("\nStrongest Negative Correlation:")
print(corr_pairs.nsmallest(1))
```

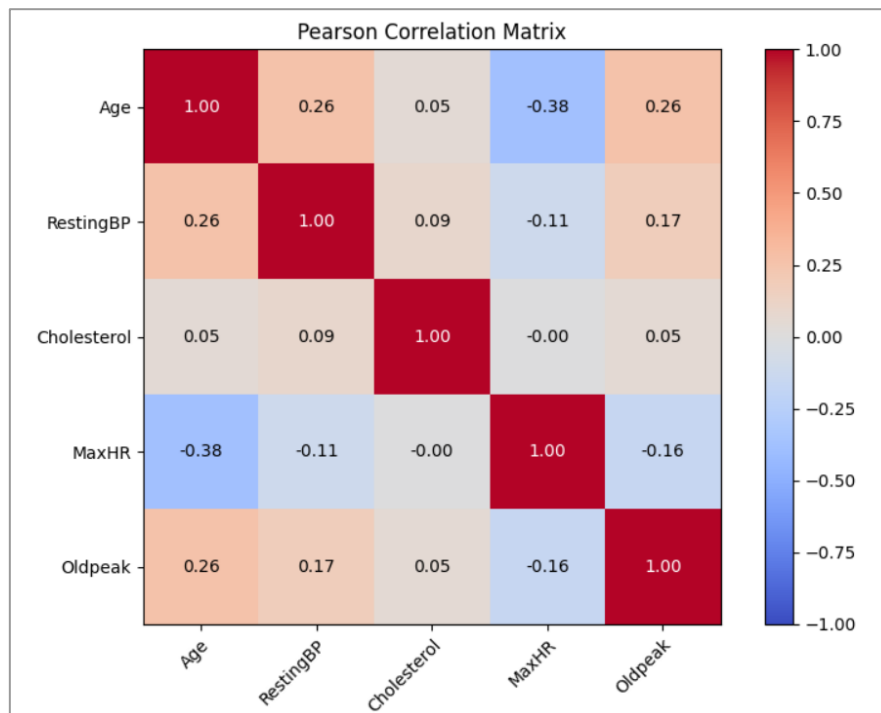


Figure 19 – Pearson Correlation Heatmap of Numerical Variables

Strongest Positive Correlation: RestingBP & Age (0.263)
 Strongest Negative Correlation: MaxHR & Age (-0.382)

Figure 18 – Strongest Positive and Negative Correlation Pairs

✓ Conclusion

- RestingBP and Age show the strongest positive correlation ($r = 0.263$), while MaxHR and Age show the strongest negative correlation ($r = -0.382$).
- All correlations among these five numerical variables are weak-to-moderate in magnitude (none exceed $|0.4|$), suggesting that no single physiological measurement alone is a strong linear predictor of another — heart disease risk is better explained by a combination of variables, as the regression models in Section 5 confirm.

4.2 ChestPainType and HeartDisease

Construct a contingency table for ChestPainType and HeartDisease. Calculate the percentage of patients with heart disease for each ChestPainType and interpret the results.

► In []: Python

```
# 1. Construct the contingency table
contingency_table = pd.crosstab(df_cleaned['ChestPainType'], df_cleaned['HeartDisease'],
                                margins=True, margins_name='Total')
print("---- Contingency Table ----")
display(contingency_table)

# 2. Calculate the percentage of heart disease (HeartDisease=1) for each type
# normalize='index' calculates percentages across the rows
pct_table = pd.crosstab(df_cleaned['ChestPainType'], df_cleaned['HeartDisease'],
                        normalize='index') * 100
print("\n---- Percentage Distribution ----")
display(pct_table.round(2).astype(str) + '%')
```

--- Contingency Table ---			
HeartDisease	0	1	Total
ChestPainType			
ASY	104	392	496
ATA	149	24	173
NAP	131	71	202
TA	26	20	46
Total	410	507	917

Figure 20 – Contingency Table

--- Percentage Distribution ---		
HeartDisease	0	1
ChestPainType		
ASY	20.97%	79.03%
ATA	86.13%	13.87%
NAP	64.85%	35.15%
TA	56.52%	43.48%

Figure 21 – Percentage Distribution

✓ Conclusion

- **ASY (Asymptomatic):** Highest risk by a wide margin — 79.03% of patients with asymptomatic chest pain are diagnosed with heart disease, and it is also the most common type recorded (496 patients).
- **TA (Typical Angina):** Moderate risk, at 43.48%.
- **NAP (Non-Anginal Pain):** About 35.15% diagnosed with heart disease.
- **ATA (Atypical Angina):** Lowest risk — only 13.87% diagnosed, meaning over 86% are perfectly normal.
- Counterintuitively, the **absence** of typical chest-pain symptoms (Asymptomatic) is the strongest indicator of heart disease in this dataset, while Atypical Angina is instead a strong indicator of a normal heart.

4.3 Test of Independence : Sex and HeartDisease

Perform a Chi-square test of independence to determine whether Sex and HeartDisease are significantly associated. State H_0 , H_1 , the Chi-square test statistic, p-value and conclusion. Use a 5% level of significance.

- **H_0 (Null Hypothesis):** Sex and Heart Disease are independent (no association between a patient's sex and their heart disease diagnosis).
- **H_1 (Alternative Hypothesis):** Sex and Heart Disease are dependent (a significant association exists).

► In []: Python

```
from scipy.stats import chi2_contingency

# 1. Create the contingency table
contingency = pd.crosstab(df_cleaned['Sex'], df_cleaned['HeartDisease'])

# 2. Perform the Chi-square test
chi2_stat, p_val, dof, expected = chi2_contingency(contingency)
print(f"Chi-square Test Statistic: {chi2_stat:.2f}")
print(f"p-value: {p_val:.2e}")
```

✓ Results & Conclusion

- Chi-square test statistic: **83.87**
- p-value: **5.28×10^{-20}**
- Since the p-value is practically zero and well below the 0.05 significance level, we **reject H0**. There is a highly significant statistical association between a patient's sex and the presence of heart disease in this dataset.

4.4 Comparison of Mean Age by Heart Disease Status

Compare the mean Age of patients with HeartDisease = 1 and HeartDisease = 0. Perform an appropriate independent samples t-test to determine whether the mean age differs significantly between the two groups. State the hypotheses, p-value and conclusion.

- **H0:** No significant difference in mean age between patients with and without heart disease ($\mu_1 = \mu_0$).
- **H1:** A significant difference exists ($\mu_1 \neq \mu_0$).

► In []: Python

```
from scipy.stats import ttest_ind

# Separate the age data into two groups based on HeartDisease status
age_hd1 = df_cleaned[df_cleaned['HeartDisease'] == 1]['Age']
age_hd0 = df_cleaned[df_cleaned['HeartDisease'] == 0]['Age']

# Calculate the mean age for each group
mean_age_hd1 = age_hd1.mean()
mean_age_hd0 = age_hd0.mean()
print(f"Mean Age of patients with Heart Disease (1): {mean_age_hd1:.2f} years")
print(f"Mean Age of patients without Heart Disease (0): {mean_age_hd0:.2f} years")

# Perform the independent samples t-test
# Using equal_var=False (Welch's t-test) is generally safer when sample sizes or
variances might differ
t_stat, p_val = ttest_ind(age_hd1, age_hd0, equal_var=False)
print(f"\nT-statistic: {t_stat:.2f}")
print(f"p-value: {p_val:.2e}")
```

✓ Results & Conclusion

- Mean Age (HeartDisease = 1): **55.90 years** | Mean Age (HeartDisease = 0): **50.55 years**
- p-value: **6.56×10^{-18}**
- Because the p-value is extremely small (< 0.05), we **reject H0**. Patients diagnosed with heart disease are significantly older on average than those without.

4.5 Comparison of Cholesterol by Sex

Compare the mean Cholesterol levels of male and female patients. Perform an appropriate independent samples t-test and interpret the result.

- **H0**: No significant difference in mean cholesterol between male and female patients ($\mu_M = \mu_F$).
- **H1**: A significant difference exists ($\mu_M \neq \mu_F$).

► In []: Python

```
from scipy.stats import ttest_ind

# Separate the cholesterol data by Sex
chol_m = df_cleaned[df_cleaned['Sex'] == 'M']['Cholesterol']
chol_f = df_cleaned[df_cleaned['Sex'] == 'F']['Cholesterol']

# Calculate means
mean_chol_m = chol_m.mean()
mean_chol_f = chol_f.mean()
print(f"Mean Cholesterol (Male): {mean_chol_m:.2f} mg/dl")
print(f"Mean Cholesterol (Female): {mean_chol_f:.2f} mg/dl")

# Perform Welch's t-test (equal_var=False) to account for different sample sizes
t_stat, p_val = ttest_ind(chol_m, chol_f, equal_var=False)
print(f"\nT-statistic: {t_stat:.2f}")
print(f"p-value: {p_val:.4f}")
```

✓ Results & Conclusion

- Mean Cholesterol (Male): **240.15 mg/dl** | Mean Cholesterol (Female): **254.70 mg/dl**
- p-value: **0.0026**
- Because the p-value is below 0.05, we **reject H0**. Female patients in this dataset have significantly higher average cholesterol than male patients.

5. Predictive Modelling

5.1 Simple Linear Regression

5.1.1 MaxHR from Age

Fit a simple linear regression model to predict MaxHR from Age. Report the regression equation, slope, intercept and R-squared value.

► In []: Python

```
from scipy.stats import linregress

# 1. Predict MaxHR from Age
res_hr = linregress(df_cleaned['Age'], df_cleaned['MaxHR'])
r_squared_hr = res_hr.rvalue**2

print(f"--- 1. MaxHR from Age ---")
print(f"Regression Equation: MaxHR = {res_hr.intercept:.2f} + ({res_hr.slope:.2f} * Age)")
print(f"Slope (b): {res_hr.slope:.2f}")
print(f"Intercept (a): {res_hr.intercept:.2f}")
print(f"R-squared: {r_squared_hr:.4f}")
```

```
--- 1. MaxHR from Age ---
Regression Equation: MaxHR = 191.99 + (-1.03 * Age)
Slope (b): -1.03
Intercept (a): 191.99
R-squared: 0.1461
```

Figure 22 – Simple Linear Regression Output — MaxHR from Age

✓ Conclusion

For every additional year of age, MaxHR is predicted to fall by about 1.03 bpm. With $R^2 = 0.1461$, Age alone explains only ~14.6% of the variation in MaxHR : a statistically real but fairly weak relationship, matching the mild downward trend already seen in the Section 3.5 scatterplot.

5.1.2 Cholesterol from RestingBP

Fit a simple linear regression model to predict Cholesterol from RestingBP. Report the regression equation, slope and R-squared value.

► In []: Python

```
# 2. Predict Cholesterol from RestingBP
res_chol = linregress(df_cleaned['RestingBP'], df_cleaned['Cholesterol'])
r_squared_chol = res_chol.rvalue**2

print(f"\n--- 2. Cholesterol from RestingBP ---")
print(f"Regression Equation: Cholesterol = {res_chol.intercept:.2f} + ({res_chol.slope:.2f} * RestingBP)")
print(f"Slope (b): {res_chol.slope:.2f}")
print(f"R-squared: {r_squared_chol:.4f}")
```

```
--- 2. Cholesterol from RestingBP ---
Regression Equation: Cholesterol = 209.35 + (0.26 * RestingBP)
Slope (b): 0.26
R-squared: 0.0074
```

Figure 23 – Simple Linear Regression Output — Cholesterol from RestingBP

✓ Conclusion

With $R^2 = 0.0074$, RestingBP explains under 1% of the variation in Cholesterol, an extremely weak relationship. This confirms what the correlation matrix in Section 4.1 already suggested ($r = 0.09$): RestingBP is not a meaningful linear predictor of Cholesterol.

5.2 Multiple Linear Regression

Fit a multiple linear regression model to predict Oldpeak using Age, RestingBP and Cholesterol as independent variables. Report the regression equation, coefficients and R-squared value.

► In []: Python

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Define the independent variables (X) and the dependent variable (y)
X = df_cleaned[['Age', 'RestingBP', 'Cholesterol']]
y = df_cleaned['Oldpeak']

# Initialize and fit the Multiple Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Extract and print the metrics
intercept = model.intercept_
coefficients = model.coef_
r_squared = model.score(X, y)

print("--- Multiple Linear Regression Results ---")
print(f"Intercept: {intercept:.4f}")
print(f"Coefficient for Age: {coefficients[0]:.4f}")
print(f"Coefficient for RestingBP: {coefficients[1]:.4f}")
print(f"Coefficient for Cholesterol: {coefficients[2]:.4f}")
print(f"R-squared: {r_squared:.4f}")
```

```
--- Multiple Linear Regression Results ---
Intercept: -1.5338
Coefficient for Age: 0.0257
Coefficient for RestingBP: 0.0066
Coefficient for Cholesterol: 0.0007
R-squared: 0.0802
```

Figure 24 – Multiple Linear Regression Results

✓ Conclusion

- Fitted equation: **Oldpeak = -1.5338 + 0.0257·Age + 0.0066·RestingBP + 0.0007·Cholesterol**
- Age has the largest (positive) effect on Oldpeak among the three predictors, while RestingBP and Cholesterol contribute very little.
- With $R^2 = 0.0802$, the three predictors jointly explain only about 8% of the variation in Oldpeak: indicating that ST-depression during exercise is driven mainly by factors not captured in this simple model (e.g. exercise intensity, ST_Slope, or ExerciseAngina).

5.3 Logistic Regression

Fit a logistic regression model to predict HeartDisease using Age, Sex, RestingBP, Cholesterol, MaxHR, Oldpeak, ChestPainType, ExerciseAngina and ST_Slope as predictors.

(a) Model Fitting

► In []: Python

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.metrics import confusion_matrix, accuracy_score

# 1. Load and clean data
df = pd.read_csv('heart_disease_data.csv')
df_cleaned = df[df['RestingBP'] > 0].copy()
median_chol = df_cleaned.loc[df_cleaned['Cholesterol'] > 0, 'Cholesterol'].median()
df_cleaned.loc[df_cleaned['Cholesterol'] == 0, 'Cholesterol'] = median_chol

# 2. Select variables
features = ['Age', 'Sex', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak',
            'ChestPainType', 'ExerciseAngina', 'ST_Slope']
X = df_cleaned[features]
y = df_cleaned['HeartDisease']

# 3. Create dummy variables for categorical features (drop_first avoids
# multicollinearity)
X_encoded = pd.get_dummies(X, drop_first=True).astype(float)
X_encoded = sm.add_constant(X_encoded) # Add intercept

# 4. (a) Fit the logistic regression model
logit_model = sm.Logit(y, X_encoded)
result = logit_model.fit()
print(result.summary())
```

Optimization terminated successfully.
Current function value: 0.346007
Iterations 7

Logit Regression Results

Dep. Variable:	HeartDisease	No. Observations:	917
Model:	Logit	Df Residuals:	904
Method:	MLE	Df Model:	12
Date:	Sun, 20 Sep 2026	Pseudo R-squ.:	0.4967
Time:	16:05:54	Log-Likelihood:	-317.29
Converged:	True	LL-Null:	-630.48
Covariance Type:	nonrobust	LLR p-value:	2.461e-126

	coef	std err	z	P> z	[0.025	0.975]
const	-1.9013	1.356	-1.402	0.161	-4.560	0.757
Age	0.0260	0.012	2.081	0.037	0.002	0.050
RestingBP	0.0021	0.006	0.354	0.723	-0.010	0.014
Cholesterol	0.0024	0.002	1.200	0.230	-0.002	0.006
MaxHR	-0.0082	0.005	-1.778	0.075	-0.017	0.001
Oldpeak	0.3406	0.114	2.998	0.003	0.118	0.563
Sex_M	1.7190	0.274	6.279	0.000	1.182	2.256
ChestPainType_ATA	-1.9932	0.314	-6.344	0.000	-2.609	-1.377
ChestPainType_NAP	-1.6284	0.252	-6.457	0.000	-2.123	-1.134
ChestPainType_TA	-1.3861	0.418	-3.317	0.001	-2.205	-0.567
ExerciseAngina_Y	0.7676	0.234	3.278	0.001	0.309	1.227
ST_Slope_Flat	0.9938	0.412	2.412	0.016	0.186	1.801
ST_Slope_Up	-1.3319	0.431	-3.091	0.002	-2.177	-0.487

Figure 25 – Logistic Regression Model Summary (statsmodels Output)

(b) Significant Variables

Using a 5% significance level ($p < 0.05$), the following predictors are statistically significant:

- **Age** ($p = 0.037$), **Oldpeak** ($p = 0.003$), **Sex_M** ($p < 0.001$), **ChestPainType_ATA** ($p < 0.001$), **ChestPainType_NAP** ($p < 0.001$), **ChestPainType_TA** ($p = 0.001$), **ExerciseAngina_Y** ($p = 0.001$), **ST_Slope_Flat** ($p = 0.016$) and **ST_Slope_Up** ($p = 0.002$).
- **RestingBP** ($p = 0.723$) and **Cholesterol** ($p = 0.230$) are not statistically significant in the presence of the other variables, their effect on heart disease appears to be captured by the other predictors.

The model as a whole is highly significant (LLR p-value $\approx 2.46 \times 10^{-126}$), with a Pseudo R^2 of 0.4967, indicating a good overall fit for a logistic model.

(c) Odds Ratios

Calculate and interpret the odds ratios for the model's predictors.

```

In [ ]: Python

# 5. (c) Calculate Odds Ratios
print("\n--- Odds Ratios ---")
print(np.exp(result.params))

```

```

--- Odds Ratios ---
const                0.149377
Age                  1.026293
RestingBP            1.002103
Cholesterol           1.002400
MaxHR                0.991810
Oldpeak              1.405768
Sex_M                5.579113
ChestPainType_ATA    0.136253
ChestPainType_NAP    0.196241
ChestPainType_TA     0.250057
ExerciseAngina_Y     2.154565
ST_Slope_Flat        2.701396
ST_Slope_Up          0.263973
dtype: float64

```

Figure 26 – Odds Ratios for the Logistic Regression Predictors

✓ Interpretation

- **Sex_M (OR = 5.58):** Being male multiplies the odds of heart disease by over 5.5 times compared to being female, holding other variables constant — the single strongest risk factor in the model.
- **ST_Slope_Flat (OR = 2.70) and ExerciseAngina_Y (OR = 2.15):** Both roughly double or triple the odds of heart disease.
- **ChestPainType_ATA (OR = 0.14), NAP (OR = 0.20) and TA (OR = 0.25):** All are strongly protective relative to the ASY baseline, confirming the pattern already observed in Section 4.2, having any of these chest pain types drastically lowers the odds of heart disease compared to being asymptomatic.
- **ST_Slope_Up (OR = 0.26):** Also strongly protective compared to a Down slope.

(d) Confusion Matrix

Generate the confusion matrix for the fitted logistic regression model (using a 0.5 classification threshold).

► In []: Python

```

# 6. (d) Confusion Matrix & (e) Accuracy
predictions_prob = result.predict(X_encoded)
predictions_binary = (predictions_prob >= 0.5).astype(int)

print("\n--- Confusion Matrix ---")
print(confusion_matrix(y, predictions_binary))
print(f"\nAccuracy: {accuracy_score(y, predictions_binary):.4f}")

```

```

--- Confusion Matrix ---
[[339  71]
 [ 57 450]]

```

Figure 27 – Confusion Matrix for the Logistic Regression Model

(e) Model Accuracy

Accuracy: 0.8604

Figure 28 – Overall Accuracy of the Logistic Regression Model

✓ Conclusion

- Of 917 patients, the model correctly classified 339 true negatives and 450 true positives, with 71 false positives and 57 false negatives.
- **Overall Accuracy: 86.04%** — a strong result, meaning the model correctly identifies the presence or absence of heart disease for roughly 86 out of every 100 patients using only routinely-collected clinical variables.

6. Overall Conclusion

This project set out to study patient health records, identify the major factors associated with heart disease, and build a working prediction model. Bringing together the results of every section above, four broad themes emerge from the analysis.

Demographic Vulnerability

Age and sex are consistently and significantly linked to heart disease throughout this study. Patients with heart disease are significantly older on average (55.90 vs 50.55 years, t-test $p \approx 6.6 \times 10^{-18}$), and sex shows both a highly significant association (Chi-square $p \approx 5.3 \times 10^{-20}$) and the single largest odds ratio in the logistic regression model (Sex_M, OR = 5.58). Men and older patients should therefore be considered priority groups for cardiovascular screening.

Symptomatic Indicators

Chest pain type and exercise-related symptoms are powerful — and sometimes counterintuitive — indicators. Asymptomatic (ASY) chest pain, rather than typical angina, carries by far the highest rate of heart disease (79.03%), while Atypical Angina is strongly protective. Similarly, exercise-induced angina and a Flat ST_Slope both roughly double the odds of heart disease. This suggests that the **absence** of classic pain symptoms should not be mistaken for the absence of risk.

Clinical Measurements

Among the continuous clinical measurements, the linear relationships are generally weak: no pair of numerical variables in the correlation matrix exceeds $|r| = 0.4$, and the simple/multiple linear regression models built on Age, RestingBP and Cholesterol achieve low R^2 values (0.146, 0.007 and 0.080 respectively). This indicates that heart disease risk is not well captured by any single continuous

measurement in isolation, and is better explained by a combination of categorical and continuous predictors together, as in the logistic regression model.

Predictive Viability

The final logistic regression model, combining demographic, symptomatic and clinical variables, achieved a strong overall accuracy of 86.04% and a Pseudo R^2 of 0.497. This confirms that a simple, interpretable statistical model built entirely from routinely-collected health records can meaningfully predict the presence of heart disease, fulfilling the project's original aim of demonstrating how statistical methods can support early identification of cardiovascular risk.

Appendix A: Tools and Libraries Used

All data cleaning, statistical analysis, visualization and modelling for this project were carried out in Python, using the following libraries:

Library	Purpose
pandas	Data loading, cleaning, grouping and tabular manipulation
numpy	Numerical computations
matplotlib	Histograms, boxplots, bar charts, scatterplots and the correlation heatmap
scipy.stats	Chi-square test, independent-samples t-tests and simple linear regression
statsmodels	Logistic regression model fitting and detailed statistical summaries
scikit-learn	Multiple linear regression, confusion matrix and accuracy score